Лабораторная работа №6.
Обработка событий в Web-форме с помощью JavaScript

Цель работы: изучить основы обработки событий в JavaScript, применить полученные знания для написания сценариев на JavaScript и HTML.

Теоретический материал

Сначала несколько вводных замечаний. События (event) и обработчики событий (event handler) относятся к JavaScript, но они скорее "встроены" в HTML-код, а не существуют самостоятельно, как те сценарии, которые мы создали. События являются встроенными, так что они не требуют команд <SCRIPT> и </SCRIPT>. Сами они являются не сценариями, а скорее небольшими интерфейсами, обеспечивающими взаимодействие между страницей и читателем.

Существует множество событий, с которыми мы со временем познакомимся, но для начала выберем одно из наиболее популярных — onMouseOver (навести курсор мыши).

Сценарий

```
<A HREF="http://www.mail.ru"
onMouseOver="window.status='Почтовая служба';
return true">Ссылка</A>
```

Все это должно быть на одной строке. Здесь он разбит на несколько строк, чтобы облегчить его просмотр.

Результат работы сценария

При наведении курсора мыши на ссылку строка состояния в окне браузера изменится.

Разбор сценария

Вы уже знаете достаточно, чтобы понять смысл написанного. Давайте быстро разберем сценарий и попробуем его как-нибудь изменить.

Во-первых, **onMouseOver** (обратите внимание на заглавные буквы) представляет собой обработчик событий (Event Handler) гипертекстовой ссылки. Это понятно? Он используется внутри гиперссылки.

Формат гипертекстовой ссылки остается без изменений. Те же команды и те же двойные кавычки. Обработчик событий **onMouseOver** ставится сразу же после адреса URL. Видите?

Событие (Event) приводится в действие, когда браузер распознает **onMouseOver="".** Общая схема уже должна быть немного понятна: два элемента, разделенные точкой. До сих пор это означало, что один является объектом, а другой методом. Но в данном случае объектом является window (окно), оно существует; status (статус) представляет собой свойство (property) окна. Это небольшой участок окна, где должен разместиться следующий далее в команде текст. Это проще запомнить, если представить, что метод обычно выражается глаголом, как write (писать) или get (получить). Свойство

выражается существительным и существует как небольшая часть элемента, стоящего перед точкой.

После **window.status** ставится знак равенства =, за которым следует то, что должно произойти. В данном случае это текст в одинарных кавычках. Он появится в строке состояния, когда вы наведете курсор на гипертекстовую ссылку.

Пожалуйста, обратите еще раз внимание на точку с запятой в конце командной строки.

Затем следует return true. Эти два слова имеют вполне определенное влияние на то, что произойдет, когда указатель мыши переместится на ссылку. Если они присутствуют, сценарий проверит, есть ли строка состояния. Если проверка будет успешной (true), происходит событие. Обратите внимание, что когда указатель мыши перемещается на ссылку, то текст в строке состояния блокируется. Он больше не изменяется при последующих перемещениях указателя мыши на ссылку. (Если обновить страницу, то можно будет это увидеть).

Другие свойства

Вернемся к свойствам, о которых мы говорили ранее. Если свойства есть у окна, то другие объекты тоже должны иметь свойства. Как насчет цвета фона? Это же свойство, не так ли? В HTML цветом фона страницы управляет команда **BGCOLOR**.

То же самое и здесь, только обязательно соблюдайте регистр символов. В JavaScript цвет фона обозначается **bgColor**. Подумаем, как создать ссылку, которая изменяла бы цвет фона окна с помощью события **onMouseOver**.

1. Во-первых, раз это ссылка, то сохраним тот же формат, который использовался ранее.
2. Что изменяется, окно или документ? Куда идет команда **bgColor**, когда создается Web-страница? В документ. Значит, это и есть нужный объект. Заменим **window** на **document**.
3. Мы собираемся изменить фоновый цвет объекта, потому заменим **status** на **bgColor**.
4. Появление текста нам больше не нужно, поэтому заменим его цветом. Возьмем розовый (**pink**).
5. Нам нужно, чтобы новый цвет оставался независимо от того, сколько раз мы будем наводить курсор на ссылку, потому вставляем **return true** после точки с запятой.

Вот что получилось ...

```
<A HREF="http://www.mail.ru"
onMouseOver="document.bgColor='pink';
return true">Щелкните здесь</A>
```

Если требуется и то, и другое

Подумаем, как можно это сделать. Здравый смысл подсказывает, что нужно написать две команды **onMouseOver.** Попробуем это реализовать. Эти две команды не разделяются. Мы хотим, чтобы они произошли

одновременно, поэтому не будем разделять команды точкой с запятой, так как точка с запятой означает конец оператора.

Новое правило: ставьте запятую, чтобы отделить друг от друга несколько событий JavaScript.

А что насчет кавычек? Запомните, в кавычки помещают отдельные элементы вроде текста. Раз нам нужно, чтобы обе команды выполнились одновременно, как одна, то ставим кавычки в самом начале первой и в самом конце второй. Таким образом, мы показываем браузеру, что все это одно событие. Понятно?

Однако нам еще могут понадобиться одинарные кавычки...

Вот что мы получим:

<A HREF="http://www.mail.ru"
onMouseOver="document.bgColor='pink',
onMouseOver=window.status='Посетите MAIL.RU';
return true">Щелкните здесь</A>

Таких обработчиков событий великое множество, и все они прекрасно работают. В следующем уроке мы поговорим еще о нескольких и научимся их комбинировать.

Вы заметили, что мы начинаем понемногу добираться до сути дела? Не забывайте, что JavaScript очень логичный язык. Позднее мы проведем специальный урок, посвященный иерархии элементов.

Давайте рассмотрим теперь действие еще нескольких обработчиков событий. Все они работают по одной схеме. Поэтому, зная формат события и представляя логику его работы, можно легко поместить его на Web-странице.

Сценарии и результаты их работы

Команда onClick

Мы знаем, что команда **onMouseOver** запускает событие, если навести курсор на ссылку. Аналогично, щелкнув по ссылке, можно с таким же успехом запустить событие с помощью команды **onClick.**

Чтобы продемонстрировать действие команды, воспользуемся методом **alert.** Этот метод использовался в задании к прошлому уроку. Вот еще раз его схема:

alert('текст, который появится в окне')

Таким образом, применяя тот же подход, что и для **onMouseOver,** получаем код:

<A HREF="http://www.mail.ru"
onClick="alert('Посмотреть почту!');">
Щелкните здесь</A>

При щелчке на ссылке появится окно с сообщением **"Посмотреть почту!"**

Помните, что внутри одинарных кавычек нельзя употреблять слова с апострофами ', иначе браузер поймет их как окончание текста, а это будет ошибка.

Команда onFocus

Это замечательный обработчик событий, который вызывает действие, когда пользователь "фокусируется" на элементе страницы. Он будет работать для элементов формы: флажков, текстовых полей, текстовых областей и др.

Вот пример:

```
<FORM>
<INPUT TYPE="text" SIZE="30"
onFocus="window.status='Текст в строке состояния';">
</FORM>
```

При использовании этого сценария выводится текстовое поле, при щелчке в котором в строке состояния выводится **`строка 'Текст в строке состояния'`**.

Команда onBlur

Если можно направить фокус на объект, значит, можно и "потерять фокус". Обработчик событий onBlur позволяет сообщить пользователю о том, что он изменил свой ответ. Этот обработчик не так часто используется, но приведем все-таки пример.

Создается текстовое поле с текстом. Надо изменить текст и затем щелкнуть мышью вне поля, имитируя переход фокуса к другому элементу.

Легко догадаться, как это делается, вот код:

```
<FORM>
<INPUT TYPE="text" SIZE="40"
VALUE="Впишите свое имя и щелкните вне текстового поля"
onBlur="alert('Вы изменили ответ — вы уверены, что он правильный?');">
</FORM>
```

Команда onChange

Действие этой команды очень похоже на действие предыдущей, onBlur. Ее главная задача – проверка. Когда мы ближе познакомимся с формами, это станет понятнее.

Этот обработчик события проверяет, сделал ли пользователь то, что от него просили. Пример очень похож на предыдущий, но действует по-другому.

```
<FORM>
<INPUT TYPE="text" SIZE="45"
VALUE="Измените текст и щелкните вне поля — затем проверьте строку состояния"
onChange="window.status='Текст был изменен'">
</FORM>
```

Т.е. при изменении текста в поле и последующем изменении фокуса в строке состояния выводится сообщение **`'Текст был изменен'`**.

Событие onSelect

Эта команда работает так же, как и три предыдущие, указывая, что в поле ввода формы произошли изменения. Отличие состоит в том, что данная команда реагирует, когда в поле ввода что-то было выделено.

Команды onLoad и onUnLoad

Пример для этих команд здесь не приводится, так как каждой из них будет посвящен отдельный урок, но они являются обработчиками событий, поэтому надо по крайней мере, их упомянуть.

Обе они помещаются внутрь команды **<BODY>** документа HTML. Они вызывают событие, когда страница открывается или закрывается, то есть когда пользователь уходит со страницы. Эти команды будут очень полезны при работе с функциями.

Объект Math

Хотя язык JavaScript создавался не для написания сложных вычислительных программ, но и в нем без математических вычислений не обойтись. Минимально достаточный набор математических функций и констант скомпонован в виде объекта **Math**, который всегда используется напрямую (экземпляры объектов для него не создаются).

Объекта Math имеет следующие свойства, которые инкапсулируют наиболее часто используемые математические константы:

Таблица 1. Свойства (математические константы) объекта Math.

Свойство	Синтаксис	Значение
E	**Math.E**	Второй замечательный предел e
LN2	**Math.LN2**	Натуральный логарифм ln2
LN10	**Math.LN10**	Натуральный логарифм ln10
LOG2E	**Math.LOG2E**	Двоичный логарифм e
LOG10E	**Math.LOG10E**	Десятичный логарифм e
PI	**Math.PI**	Число «пи»
SQRT1_2	**Math.SQRT1_2**	Корень квадратный из 1/2
SQRT2	**Math.SQRT2**	Корень квадратный из 2

Кроме математических констант, методы объекта Math представляют (инкапсулируют) наиболее часто используемые математические функции:

Таблица 2. Методы (математические функции) объекта Math.

Метод	Синтаксис	Значение
abs()	**Math.abs(x)**	Абсолютная величина числа x
acos()	**Math.acos(x)**	arccoc(x)
asin()	**Math.asin(x)**	arcsin(x)
atan()	**Math.atan(x)**	arctan(x)
ceil()	**Math.ceil(x)**	Округление до меньшего целого
cos()	**Math.cos(x)**	cos(x)
exp()	**Math.exp(x)**	e^x
floor()	**Math.floor(x)**	Округление до большего целого
log()	**Math.log(x)**	Натуральный логарифм
max()	**Math.max(x,y)**	Максимум двух чисел
min()	**Math.min(x,y)**	Минимум двух чисел
pow()	**Math.pow(x,y)**	x^y
random()	**Math.random(x)**	Случайное число от 0 до 1
round()	**Math.round(x)**	Округление до ближайшего целого

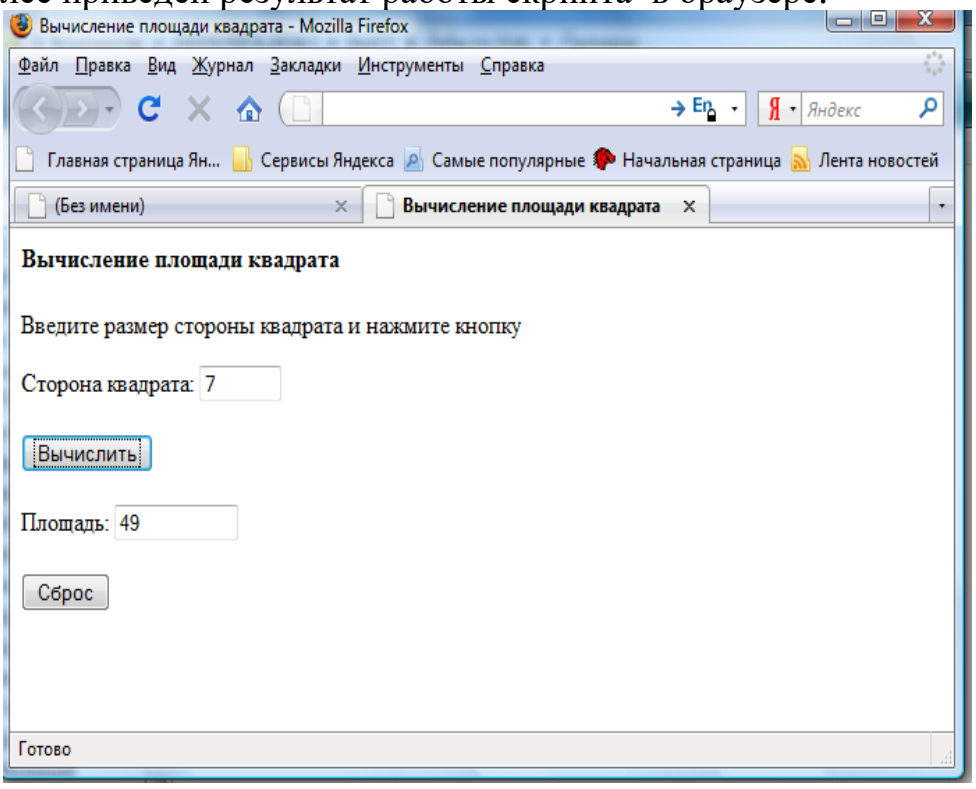
sin()	**Math.sin(x)**	sin(x)
sqrt()	**Math.sqrt(x)**	Корень квадратный их x
tan()	**Math.tan(x)**	Тангенс x

Пример: создать форму, для вычисления площади квадрата. Площадь вычисляется в результате щелчка по кнопке.

Разработанный HTML – код приведен ниже:

```
<html>
<head>
 <title> Вычисление площади квадрата </title>
</head>
<body>
 <h4>Вычисление площади квадрата</h4>
 <form name ="form1">
 <p> Введите размер стороны квадрата и нажмите кнопку </p>
  Сторона квадрата:
  <input type="text" name="num" size =5> <br><br>
  <input type="button" value="Вычислить"  onClick=
"document.form1.res.value=Math.pow(document.form1.num.value,2)">
<br><br>
  Площадь:
  <input type="text" name="res" size=10>
 <br><br>
  <input type="reset">
  </form>
</body>
</html>
```

Далее приведен результат работы скрипта в браузере:



Общая постановка задачи

1. Выполнить индивидуальное задание по варианту.
2. Поместить разработанный **JS**-код сценария на FTP-сервере.
3. Составить электронный отчет о проделанной работе. На титульном листе отчета указать URL для запуска разработанной программы на выполнение.

Список индивидуальных данных

1. Создать форму, для вычисления площади треугольника по трем сторонам. Площадь вычисляется в результате щелчка по кнопке с параметрами type=”button”.
2. Создать форму, для вычисления площади параллелограмма по двум сторонам и углу между ними. Площадь вычисляется в результате потери фокуса у последнего текстового поля формы.
3. Создать форму, для вычисления объема пирамиды по площади основания и высоте. Площадь вычисляется в результате изменения значения последнего текстового поля формы.
4. Создать форму, для вычисления длины гипотенузы по катету и противолежащему углу . Длина вычисляется в результате щелчка по кнопке с параметрами type= “submit”.
5. Создать форму, для вычисления расстояния между двумя точками. Длина вычисляется в результате выделения текста в первом текстовом поле.
6. Создать форму, для вычисления площади прямоугольного треугольника. Площадь вычисляется в результате потери фокуса у последнего текстового поля.
7. Создать форму, ввести значение трафика Интернет в Кб. Вычислить трафик в КБ в результате получения фокуса у второго текстового поля.
8. Создать форму, для вычисления площади трапеции. Площадь вычисляется в результате изменения данных в последнем текстовом поле.
9. Создать форму, для вычисления площади прямоугольника. Площадь вычисляется в результате щелчка по последнему текстовому полю.
10. Создать форму, для вычисления площади круга. Площадь вычисляется в результате щелчка по ссылке.
11. Создать форму, для вычисления длины окружности. Длина вычисляется в результате выделения данных текстового поля.
12. Создать форму, для вычисления периметра квадрата. Периметр вычисляется в результате изменения данных в последнем текстовом поле.
13. Создать форму, для вычисления периметра треугольника. Периметр вычисляется в результате наведения мышкой на гиперссылку.
14. Создать форму, для вычисления объема цилиндра. Объем вычисляется в результате отведения мышки от гиперссылки.
15. Создать форму, для вычисления объема конуса. Объем вычисляется в результате наведения мышки на гиперссылку.
16. Создать форму, для вычисления объема цилиндра. Объем вычисляется в результате щелчка мышкой по кнопке.

17. Создать форму, для вычисления объема куба. Объем вычисляется в результате при получении фокуса 1-м текстовым полем.

18. Создать форму, для вычисления площади боковой поверхности пирамиды. Площадь вычисляется при изменении значения текстового поля.

19. Создать форму, для вычисления площади боковой поверхности призмы. Площадь вычисляется в результате выделения в текстовом поле.

20. Создать форму, для вычисления площади n-го члена арифметической прогрессии. Вычисление происходит в результате наведения мыши на гиперссылку текстовом поле.

21. Создать форму, для вычисления среднего балла за сессию. Вычисление происходит в результате щелчка мышью на гиперссылку текстовом поле.

22. Создать форму, для вычисления среднего арифметического трех введенных чисел. Вычисление происходит в результате изменения значения третьего числа.

23. Создать форму, ввести названия и стоимость трех купленных в последнее время книг. Вычислить стоимость трех книг при нажатии на кнопку.

24. Создать форму, ввести расстояние от вашего дома до университета и время, затрачиваемое вами на преодоление этого пути. Вычислить скорость вашего движения после ввода времени.

25. Создать форму, ввести значение температуры в помещении в Цельсии. Определите значение температуры в Фаренгейт при потере фокуса.